

Федеральное государственное автономное образовательное учреждение высшего
образования

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет информационных технологий

Кафедра «Информатика и информационные технологии»

Направление подготовки/ специальность: Информационные системы и технологии

ОТЧЕТ

по проектной практике

Студент: Зверев Савелий Максимович Группа: 251-331

Место прохождения практики: Московский Политех, кафедра информатика и
информационные технологии

Отчет принят с оценкой _____ Дата _____

Руководитель практики: _____

Москва 2026

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
1. Общая информация о проекте.....	3
2. Общая характеристика деятельности организации	4
3. Описание задания по проектной практике	5
4. Описание достигнутых результатов по проектной практике	5
ЗАКЛЮЧЕНИЕ	8
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ	8
ПРИЛОЖЕНИЯ	9

ВВЕДЕНИЕ

Ежегодно более 3000 студентов Московского Политеха выбирают темы выпускных квалификационных работ и проектов, однако отсутствие цифрового инструмента для поиска научного руководителя создаёт хаос, приводит к перегрузке преподавателей и значительным потерям времени. Информация о научных интересах преподавателей, их актуальной занятости и темах проектов, которые они готовы вести, остаётся разрозненной, а прозрачные критерии для обоснованного выбора, такие как отзывы студентов или рейтинг, отсутствуют. В связи с этим назрела необходимость в создании единого прозрачного сервиса, который позволит автоматизировать процесс сбора заявок, учёта нагрузки преподавателей, а также предоставит студентам эффективные инструменты для поиска и коммуникации. Проект «Филин» направлен на решение этих задач путём разработки приложения, обеспечивающего удобный и прозрачный подбор научного руководителя. В рамках проекта планируется создать единую базу преподавателей с подробными профилями, внедрить систему фильтров и поиска, реализовать встроенный чат для оперативной связи, а также систему отзывов и рейтинга для повышения качества взаимодействия и снижения административной нагрузки.

1. Общая информация о проекте

Название проекта: Филин

Цель: Разработать и внедрить приложение, обеспечивающее эффективный и прозрачный подбор научного руководителя.

Задачи:

1. Создать единую базу преподавателей с профилями, где указаны: образование, научные интересы, темы проектов и текущая занятость.
2. Разработать систему фильтров и поиска для быстрого подбора руководителя по факультету, направлениям и формату работы
3. Реализовать встроенный чат для оперативной коммуникации студентов и преподавателей.

4. Внедрить систему отзывов и рейтинга для повышения качества взаимодействия и прозрачности выбора.
5. Автоматизировать процесс сбора заявок и учета нагрузки преподавателей для снижения административной нагрузки.

2. Общая характеристика деятельности организации

Наименование заказчика

Центр проектной деятельности Мосполитеха (ЦПД)

Организационная структура

Заказчик выступает как координатор проектной деятельности студентов. Непосредственный контроль осуществляет руководитель практики и куратор по дисциплине «Проектная деятельность». В процессе выполнения проекта команда студентов взаимодействовала с заказчиком для уточнения требований и получения обратной связи.

Описание деятельности

Основная деятельность заказчика – образовательная и научно-исследовательская. Применительно к данному проекту – формирование у студентов компетенций в области проектирования, веб-разработки и документирования, а также получение готового дизайн-проекта, который может быть использован в учебных и выставочных целях.

3. Описание задания по проектной практике

Задание по проектной практике состояло из двух частей: базовой (обязательной для всех) и вариативной.

Базовая часть задания включала:

1. Настройку Git и создание репозитория на GitHub/GitVerse.
2. Освоение базовых команд Git (clone, commit, push, branch).
3. Ведение документации в формате Markdown.
4. Создание статического веб-сайта (HTML/CSS) с описанием проекта, участников, журнала работы и ресурсов.
5. Написание отчёта о взаимодействии с организацией-партнёром.
6. Подготовку итогового отчёта в DOCX и PDF.

Вариативная часть задания:

1. Создать игру с использованием библиотеки Pygame на языке Python.
2. Разработать RPG с элементами: игрок, NPC (дружественные и враждебные), система инвентаря, квесты, торговля, боевая система.
3. Реализовать анимацию персонажа, взаимодействие с объектами (сундук, монеты), интерфейс пользователя.
4. Задokumentировать процесс разработки в Markdown, подготовить техническое руководство.

Форма отчётности: код в репозитории, статический сайт, текст отчёта.

4. Описание достигнутых результатов по проектной практике

В ходе выполнения проектной практики полностью выполнены все задачи базовой и вариативной частей задания.

Выполнение базовой части задания:

1. Создан репозиторий на GitHub, освоены базовые команды (clone, commit, push, branch)

2. Разработан сайт на HTML/CSS, содержащий все обязательные страницы: главная, о проекте, участники, журнал, ресурсы, каждый внёс свой вклад:
Я разработал почти полностью весь сайт, а Варданын Людвиг Леонович сделал страницу: ресурсы.
3. Был составлен отчёт о посещении Дня карьеры «Ростелеком»
4. Был изучен синтаксис Markdown

Выполнение вариативной части

Разработана полноценная RPG-игра на языке программирования Python с использованием библиотеки Pygame. Чтобы пройти игру необходимо: уничтожить всех крыс. Работа распределялась между двумя разработчиками следующим образом:

Мой вклад в игру (бэкенд / игровая логика)

Компонент	Что сделано
Класс Player	Инициализация, анимация движения (бег и покой), методы move, draw, характеристики (уровень, HP, атака, защита, опыт), система инвентаря и экипировки (equip_item, unequip, use_item), gain_experience, level_up
Класс NPC	Базовый класс, характеристики врагов, диалоги
Класс Item	Атрибуты предметов, метод use
Класс Quest	Полная реализация квестов (цели, прогресс, награды, проверка завершения)
Игровой цикл (main.py)	Движение, границы, обработка взаимодействия с NPC, запуск боя, система квестов, сбор монеты, инициализация объектов

Варданын Людвиг Леонович (фронтенд / интерфейсы)

Компонент	Что сделано
show_dialogue	Всплывающие диалоговые окна персонажей
show_victory	Окно победы с градиентом, звёздами и выбором выхода
show_pause_menu	Меню паузы с продолжением или выходом
show_inventory	Полноценный инвентарь: сетка предметов, иконки, подсказки при наведении мыши, экипировка/снятие предметов
show_battle_inventory	Упрощённый инвентарь для боя (только расходимые предметы)
show_chest	Окно сундука с сеткой предметов, иконками и подсказками
trade_with_trader	Торговля с NPC: меню товаров, подсветка выбранного, подсказки
start_combat	Полностью переработанная боевая система: меню (АТАКА, РЮКЗАК, СТАТУС, УБЕЖАТЬ), полосы HP, увеличенные спрайты (120×120), управление A/D, диалоговое окно, статус врага
HUD	Полоска здоровья с цифрами внутри, иконка золота, отображение активного квеста
Методы get_icon и get_effect_string в классе Item	Визуальное отображение иконок и подсказок эффектов

ЗАКЛЮЧЕНИЕ

В результате выполнения проектной практики была разработана полноценная RPG-игра на библиотеке Pygame.

В процессе разработки было реализовано 4 основных класса: Player (игрок), NPC (неигровые персонажи), Item (предметы) и Quest (квесты). На игровой карте размещено 5 NPC, включая торговца, старейшину и трёх враждебных крыс.

Боевая система является пошаговой и предлагает игроку на выбор 4 варианта действий: атака, использование предметов из рюкзака, просмотр статуса врага или побег. В игре реализовано более 6 типов предметов, включая зелья здоровья, оружие, броню и золото.

Также в игре присутствует квест на уничтожение трёх крыс с выдачей награды в виде золота и опыта при успешном выполнении. Параллельно с игрой был разработан статический сайт проекта, содержащий 5 обязательных страниц: главную, страницу о проекте, страницу участников, журнал работ и страницу с полезными ресурсами.

Также благодаря проектной деятельности ключевым результатом моего личного развития стало изучение фреймворка **Flutter** для кроссплатформенной разработки мобильных приложений.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Документация Pygame – Текст: электронный // PYGAME.ORG: [сайт]. – URL: <https://www.pygame.org/docs/> (дата обращения: 11.05.2026)
2. Руководство по Markdown – Текст: электронный // RU.HEXLET.IO: [сайт]. – URL: <https://ru.hexlet.io/blog/posts/что-такое-markdown-i-zachem-on-nuzhen> (дата обращения: 11.05.2026)

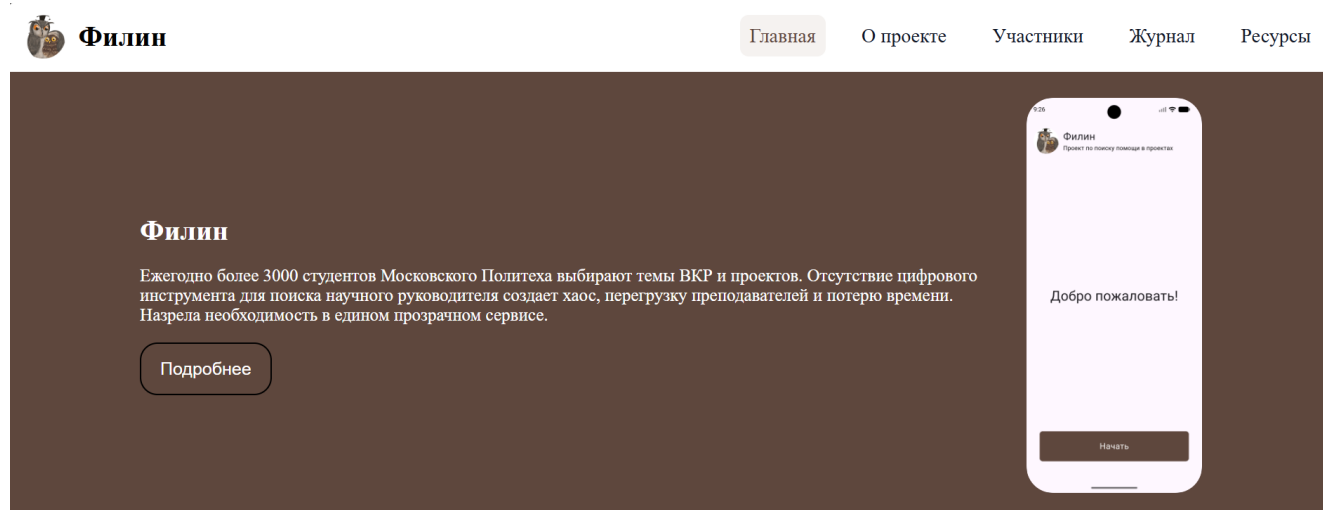
3. Документация по Git – Текст: электронный // GIT-SCM.COM: [сайт]. – URL: <https://git-scm.com/docs> (дата обращения: 11.05.2026)
4. Джон Дакетт, HTML и CSS. Разработка и дизайн веб-сайтов : учеб. пособие / Джон Дакетт – Москва : Эксмо, 2024. – 480 с. – ISBN 978-5-04-101286-1 – Текст : непосредственный. (дата обращения: 11.05.2026)

ПРИЛОЖЕНИЯ

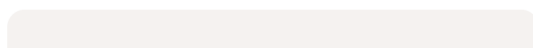
ПРИЛОЖЕНИЕ А

Ссылка на GitHub: <https://github.com/Savely00001/project-practice.git>

ПРИЛОЖЕНИЕ Б. Скриншоты страниц веб-сайта (домашняя, о проекте, участники, журнал, ресурсы), а также игры.



Цель



Цель

Разработать и внедрить приложение, обеспечивающее эффективный и прозрачный подбор научного руководителя.

Ключевые задачи



1. Создать единую базу преподавателей с профилями, где указаны: образование, научные интересы, темы проектов и текущая занятость.
2. Разработать систему фильтров и поиска для быстрого подбора руководителя по факультету, направлениям и формату работы.



1. Реализовать встроенный чат для оперативной коммуникации студентов и преподавателей.
2. Внедрить систему отзывов и рейтинга для повышения качества взаимодействия и прозрачности выбора.



1. Автоматизировать процесс сбора заявок и учета нагрузки преподавателей для снижения административной нагрузки.



Филин

[Главная](#)

[О проекте](#)

[Участники](#)

[Журнал](#)

[Ресурсы](#)

Обзор проекта

Проблематика

Факторы, снижающие эффективность подбора

1. Долгий и хаотичный ручной поиск
2. Нет единого цифрового инструмента: информация о преподавателях разрознена
3. Непонятно, кто какими научными интересами обладает и какие темы может вести
4. Отсутствуют прозрачные критерии для выбора (отзывы, занятость, формат работы)
5. Высокая нагрузка на преподавателей и администрацию в пиковые периоды



Актуальность

Ежегодно более 3000 студентов Московского Политеха выбирают темы ВКР и проектов. Отсутствие цифрового инструмента для поиска научного руководителя создает хаос, перегрузку преподавателей и потерю времени. Назрела необходимость в едином прозрачном сервисе.



Полезные ресурсы и материалы

[Скачать общую инструкцию](#)

[Ссылка на опрос](#)

Филин

Сервис для подбора научного руководителя ВКР и проектов

Быстрый доступ

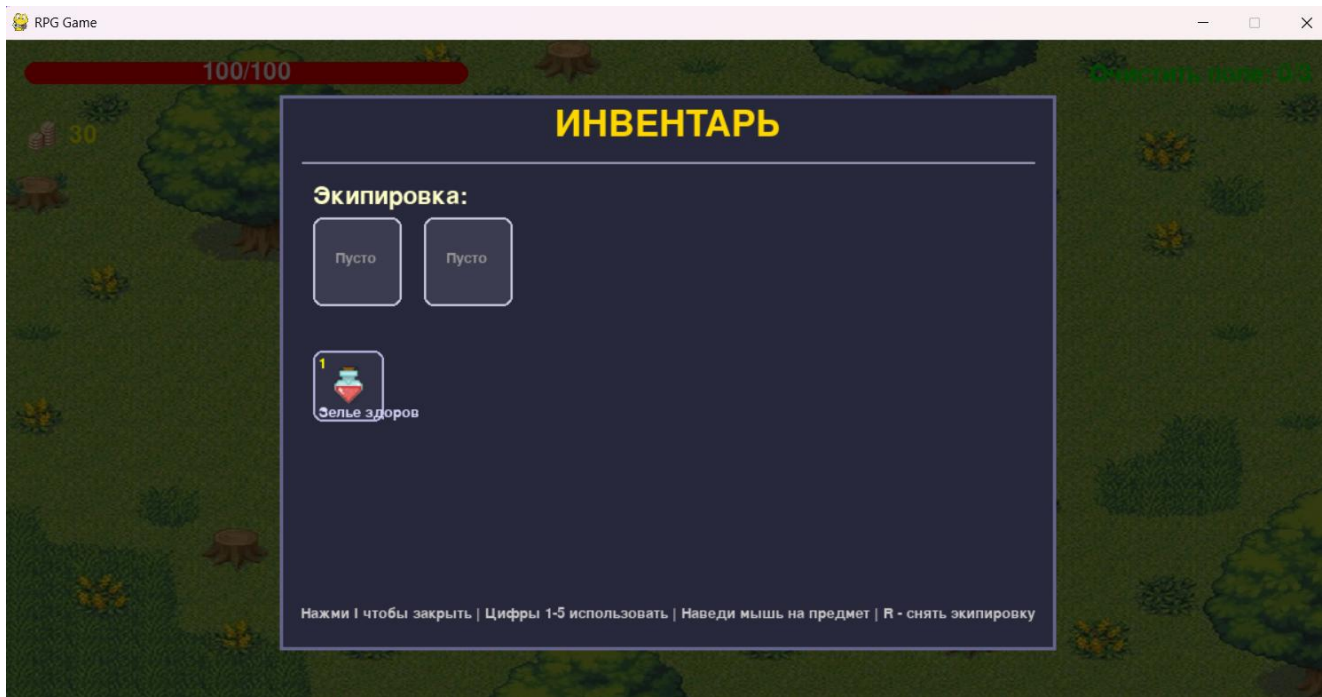
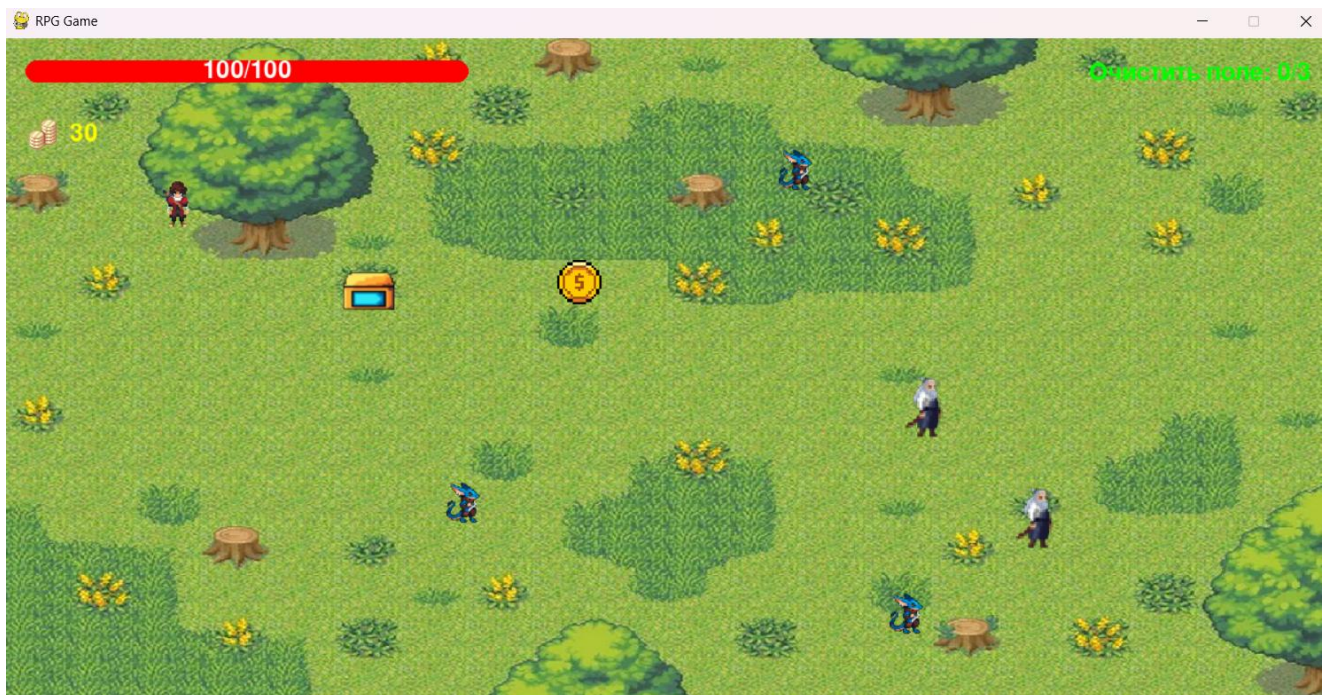
[Главная](#)
[О проекте](#)
[Участники](#)
[Журнал](#)
[Ресурсы](#)

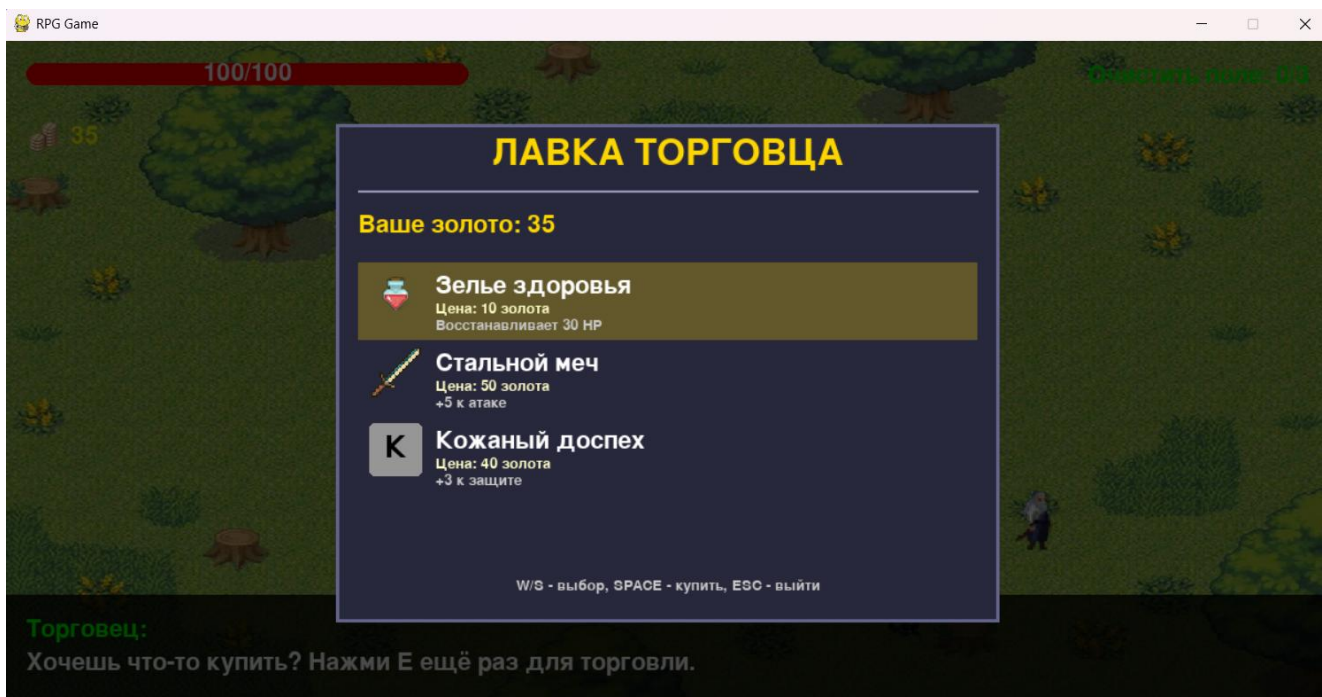
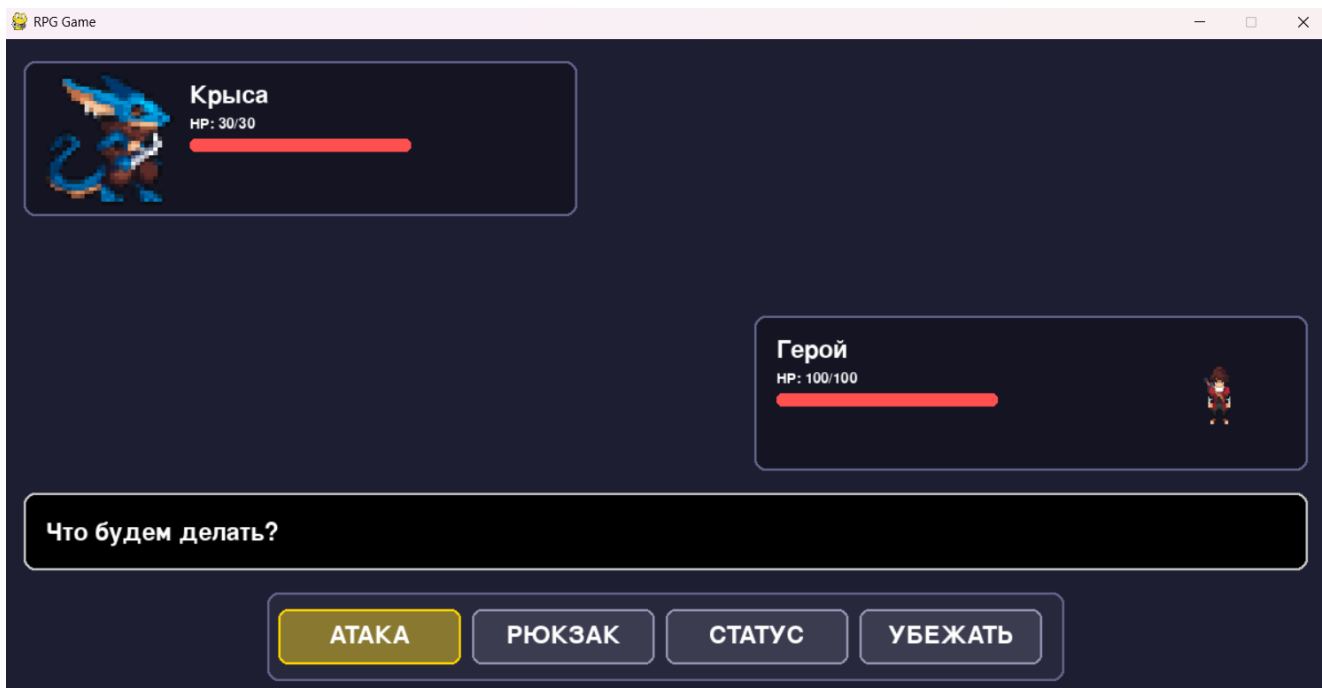
Контакты

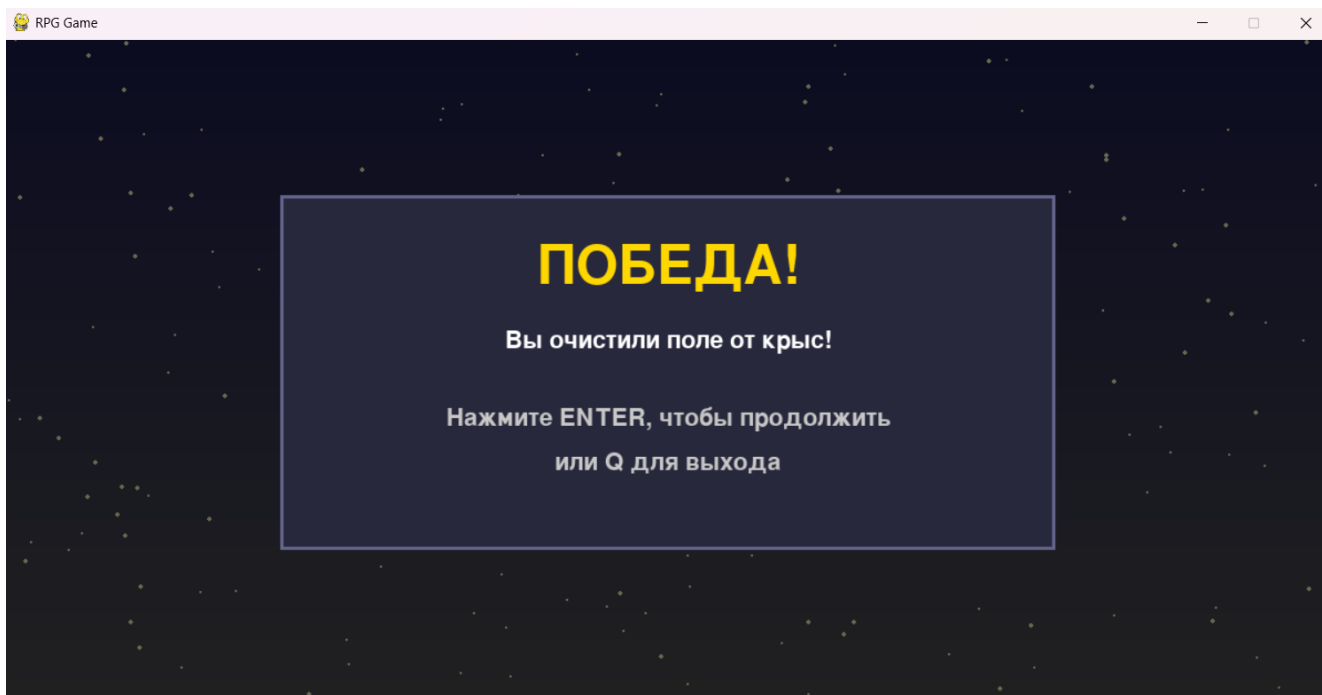
Email: XXXXXXX@gmail.com
Телефон: +7XXXXXXX

© 2026 Филин. Все права защищены.

Игра







ПРИЛОЖЕНИЕ В. Листинг кода игры

```
import pygame
import sys
import time
import os
import random

# ===== КЛАСС ИГРОКА =====
class Player:
    def __init__(self, x, y, scale=1.0):
        self.frame_width = 96
        self.frame_height = 80
        self.width = int(self.frame_width * scale)
        self.height = int(self.frame_height * scale)
        self.x = x
        self.y = y
        self.speed = 5
        self.direction = 'down'
        self.is_moving = False

        self.name = "Герой"
        self.level = 1
        self.max_hp = 100
        self.hp = 100
        self.base_attack = 10
        self.base_defense = 5
        self.attack = self.base_attack
        self.defense = self.base_defense
        self.experience = 0
        self.inventory = []
        self.gold = 30
```



```

self.active_quests = []

self.equipped_weapon = None
self.equipped_armor = None

self.FRAMES = 8
self.run_anim = {}
self.idle_anim = {}
directions = ['down', 'up', 'left', 'right']
for d in directions:
    path = f'assets/player_{d}_sheet.png'
    try:
        sheet = pygame.image.load(path).convert_alpha()
    except:
        sheet = pygame.Surface((self.frame_width, self.frame_height))
        sheet.fill((255, 0, 255))
    frames = []
    for i in range(self.FRAMES):
        rect = pygame.Rect(i * self.frame_width, 0, self.frame_width,
self.frame_height)
        frame = sheet.subsurface(rect)
        if scale != 1.0:
            frame = pygame.transform.scale(frame, (self.width, self.height))
        frames.append(frame)
    self.run_anim[d] = frames

    path_idle = f'assets/idle_{d}.png'
    try:
        sheet_idle = pygame.image.load(path_idle).convert_alpha()
    except:
        sheet_idle = pygame.Surface((self.frame_width, self.frame_height))
        sheet_idle.fill((0, 255, 0))
    frames_idle = []
    for i in range(self.FRAMES):
        rect = pygame.Rect(i * self.frame_width, 0, self.frame_width,
self.frame_height)
        frame = sheet_idle.subsurface(rect)
        if scale != 1.0:
            frame = pygame.transform.scale(frame, (self.width, self.height))
        frames_idle.append(frame)
    self.idle_anim[d] = frames_idle

self.current_frame = 0
self.anim_speed_run = 0.08
self.anim_speed_idle = 0.15
self.last_update = pygame.time.get_ticks()
self.rect = pygame.Rect(x, y, self.width, self.height)

def update_animation(self):
    now = pygame.time.get_ticks()

```

```

        speed = self.anim_speed_run if self.is_moving else self.anim_speed_idle
        if now - self.last_update > speed * 1000:
            self.last_update = now
            self.current_frame = (self.current_frame + 1) % self.FRAMES

def move(self, dx, dy):
    old_dir = self.direction
    old_moving = self.is_moving
    if dx > 0:
        self.direction = 'right'
    elif dx < 0:
        self.direction = 'left'
    elif dy > 0:
        self.direction = 'down'
    elif dy < 0:
        self.direction = 'up'
    self.is_moving = (dx != 0 or dy != 0)
    if self.direction != old_dir or self.is_moving != old_moving:
        self.current_frame = 0
        self.last_update = pygame.time.get_ticks()
    self.x += dx
    self.y += dy
    self.rect.x = self.x
    self.rect.y = self.y

def draw(self, screen):
    self.update_animation()
    current = self.run_anim if self.is_moving else self.idle_anim
    sprite = current[self.direction][self.current_frame]
    screen.blit(sprite, (self.x, self.y))

def recalc_stats(self):
    self.attack = self.base_attack
    self.defense = self.base_defense
    if self.equipped_weapon:
        self.attack += self.equipped_weapon.effect
    if self.equipped_armor:
        self.defense += self.equipped_armor.effect

def equip_item(self, item):
    if item.type == 'weapon':
        if self.equipped_weapon == item:
            return False
        if self.equipped_weapon:
            self.inventory.append(self.equipped_weapon)
        self.equipped_weapon = item
    if item in self.inventory:
        self.inventory.remove(item)
    self.recalc_stats()
    return True

```



```

    elif item.type == 'armor':
        if self.equipped_armor == item:
            return False
        if self.equipped_armor:
            self.inventory.append(self.equipped_armor)
        self.equipped_armor = item
        if item in self.inventory:
            self.inventory.remove(item)
        self.recalc_stats()
        return True
    return False

def unequip(self, slot):
    if slot == 'weapon' and self.equipped_weapon:
        self.inventory.append(self.equipped_weapon)
        self.equipped_weapon = None
        self.recalc_stats()
        return True
    elif slot == 'armor' and self.equipped_armor:
        self.inventory.append(self.equipped_armor)
        self.equipped_armor = None
        self.recalc_stats()
        return True
    return False

def use_item(self, item):
    if item.type == 'potion':
        self.hp = min(self.hp + item.effect, self.max_hp)
        return True
    elif item.type in ('weapon', 'armor'):
        return self.equip_item(item)
    return False

def gain_experience(self, amount):
    self.experience += amount
    if self.experience >= self.level * 100:
        self.level_up()

def level_up(self):
    self.level += 1
    self.max_hp += 20
    self.hp = self.max_hp
    self.base_attack += 2
    self.base_defense += 1
    self.recalc_stats()
    self.experience = 0
    print(f"Уровень повышен! Теперь {self.level}")

```

===== КЛАСС NPC =====

class NPC:

```

def __init__(self, x, y, name, sprite_path, is_enemy=False, scale=1.0):
    self.x = x
    self.y = y
    self.name = name
    self.is_enemy = is_enemy
    try:
        img = pygame.image.load(sprite_path).convert_alpha()
    except:
        img = pygame.Surface((32, 32))
        img.fill((128, 128, 128))
    w = int(img.get_width() * scale)
    h = int(img.get_height() * scale)
    self.sprite = pygame.transform.scale(img, (w, h)) if scale != 1.0 else img
    self.width = self.sprite.get_width()
    self.height = self.sprite.get_height()
    self.rect = pygame.Rect(x, y, self.width, self.height)
    if is_enemy:
        self.hp = 30
        self.max_hp = 30
        self.attack = 5
        self.defense = 1
        self.exp_reward = 15
        self.gold_reward = 5
    self.dialogues = []

def draw(self, screen):
    screen.blit(self.sprite, (self.x, self.y))

def add_dialogue(self, text):
    self.dialogues.append(text)

def get_dialogue(self):
    return self.dialogues[0] if self.dialogues else "..."

```

===== КЛАСС ПРЕДМЕТА =====

```

class Item:
    def __init__(self, name, desc, item_type, value, effect, icon_path=None):
        self.name = name
        self.desc = desc
        self.type = item_type
        self.value = value
        self.effect = effect
        self.icon_path = icon_path
        self._icon = None

    def get_icon(self, size=32):
        if self._icon:
            return pygame.transform.scale(self._icon, (size, size))
        if self.icon_path and os.path.exists(self.icon_path):
            try:

```

```

        img = pygame.image.load(self.icon_path).convert_alpha()
        self._icon = pygame.transform.scale(img, (size, size))
        return self._icon
    except:
        pass

    surf = pygame.Surface((size, size), pygame.SRCALPHA)
    color_map = {'potion': (255, 50, 50), 'weapon': (200, 200, 255), 'armor':
(150, 150, 150), 'gold': (255, 215, 0)}
    color = color_map.get(self.type, (200, 200, 200))
    pygame.draw.rect(surf, color, (0, 0, size, size), border_radius=5)
    font = pygame.font.Font(None, size - 8)
    letter = self.name[0].upper() if self.name else "?"
    text = font.render(letter, True, (0, 0, 0))
    surf.blit(text, (size // 2 - text.get_width() // 2, size // 2 -
text.get_height() // 2))
    self._icon = surf
    return surf

def get_effect_string(self):
    if self.type == 'potion':
        return f"Восстанавливает {self.effect} HP"
    elif self.type == 'weapon':
        return f"+{self.effect} к атаке"
    elif self.type == 'armor':
        return f"+{self.effect} к защите"
    elif self.type == 'gold':
        return f"Добавляет {self.effect} золота"
    else:
        return self.desc

# ===== КЛАСС КВЕСТА =====
class Quest:
    def __init__(self, name, desc, gold_reward, exp_reward):
        self.name = name
        self.desc = desc
        self.completed = False
        self.gold_reward = gold_reward
        self.exp_reward = exp_reward
        self.objectives = []

    def add_objective(self, description, target):
        self.objectives.append({'desc': description, 'target': target, 'current': 0})

    def update_objective(self, index, amount=1):
        if index < len(self.objectives):
            self.objectives[index]['current'] += amount
            if all(obj['current'] >= obj['target'] for obj in self.objectives):
                self.completed = True
            return self.completed
        return False

```

```

def claim_reward(self, player):
    if self.completed:
        player.gold += self.gold_reward
        player.gain_experience(self.exp_reward)
        return True
    return False

# ===== ВСПОМОГАТЕЛЬНЫЕ ФУНКЦИИ =====
def show_dialogue(screen, font, text, speaker):
    overlay = pygame.Surface((1200, 100))
    overlay.set_alpha(200)
    overlay.fill((0, 0, 0))
    screen.blit(overlay, (0, 500))
    name_surf = font.render(f"{speaker}:", True, (0, 255, 0))
    txt_surf = font.render(text, True, (255, 255, 255))
    screen.blit(name_surf, (20, 520))
    screen.blit(txt_surf, (20, 550))
    pygame.display.flip()
    pygame.time.wait(2000)

def show_victory(screen, font):
    """Красивое окно победы с градиентом, звёздами и выбором выхода"""
    # Градиентный фон
    victory_surface = pygame.Surface((1200, 600))
    for y in range(600):
        color_val = 30 + int(y / 600 * 50)
        pygame.draw.line(victory_surface, (color_val, color_val, 80), (0, y), (1200,
y))
    screen.blit(victory_surface, (0, 0))

    # Звёзды
    for _ in range(200):
        x = random.randint(0, 1200)
        y = random.randint(0, 600)
        pygame.draw.circle(screen, (255, 255, 200), (x, y), random.randint(1, 2))

    # Затемнение
    overlay = pygame.Surface((1200, 600), pygame.SRCALPHA)
    overlay.fill((0, 0, 0, 150))
    screen.blit(overlay, (0, 0))

    # Панель (увеличим ширину до 700)
    victory_width = 700
    victory_height = 320
    victory_x = (1200 - victory_width) // 2
    victory_y = (600 - victory_height) // 2

    panel = pygame.Surface((victory_width, victory_height))
    panel.fill((40, 40, 60))

```

```

pygame.draw.rect(panel, (100, 100, 140), panel.get_rect(), 3)
screen.blit(panel, (victory_x, victory_y))

title_font = pygame.font.Font(None, 72)
title = title_font.render("ПОБЕДА!", True, (255, 215, 0))
screen.blit(title, (victory_x + victory_width // 2 - title.get_width() // 2,
victory_y + 40))

text = font.render("Вы очистили поле от крыс!", True, (255, 255, 255))
screen.blit(text, (victory_x + victory_width // 2 - text.get_width() // 2,
victory_y + 120))

# Разбиваем длинную строку на две
line1 = "Нажмите ENTER, чтобы продолжить"
line2 = "или Q для выхода"
text1 = font.render(line1, True, (200, 200, 200))
text2 = font.render(line2, True, (200, 200, 200))
screen.blit(text1, (victory_x + victory_width // 2 - text1.get_width() // 2,
victory_y + 190))
screen.blit(text2, (victory_x + victory_width // 2 - text2.get_width() // 2,
victory_y + 230))

pygame.display.flip()

waiting = True
while waiting:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_RETURN:
                waiting = False
            if event.key == pygame.K_q:
                pygame.quit()
                sys.exit()
    pygame.time.wait(50)

def show_pause_menu(screen, font, small_font):
    background_copy = screen.copy()
    paused = True
    while paused:
        screen.blit(background_copy, (0, 0))
        overlay = pygame.Surface((1200, 600), pygame.SRCALPHA)
        overlay.fill((0, 0, 0, 200))
        screen.blit(overlay, (0, 0))

        menu_width = 500
        menu_height = 250
        menu_x = (1200 - menu_width) // 2

```

```

    menu_y = (600 - menu_height) // 2
    panel = pygame.Surface((menu_width, menu_height))
    panel.fill((40, 40, 60))
    pygame.draw.rect(panel, (100, 100, 140), panel.get_rect(), 3)
    screen.blit(panel, (menu_x, menu_y))

    title_font = pygame.font.Font(None, 64)
    title = title_font.render("ПАУЗА", True, (255, 215, 0))
    screen.blit(title, (menu_x + menu_width // 2 - title.get_width() // 2, menu_y
+ 30))

    continue_text = font.render("Продолжить (ESC / Enter)", True, (255, 255,
255))
    quit_text = font.render("Выйти из игры (Q)", True, (255, 255, 255))
    screen.blit(continue_text, (menu_x + menu_width // 2 -
continue_text.get_width() // 2, menu_y + 110))
    screen.blit(quit_text, (menu_x + menu_width // 2 - quit_text.get_width() //
2, menu_y + 160))

    hint = small_font.render("Нажми ESC или Enter для продолжения, Q для выхода",
True, (200, 200, 200))
    screen.blit(hint, (menu_x + menu_width // 2 - hint.get_width() // 2, menu_y +
210))

    pygame.display.flip()

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_ESCAPE or event.key == pygame.K_RETURN:
                paused = False
            if event.key == pygame.K_q:
                pygame.quit()
                sys.exit()
    pygame.time.wait(50)

def show_inventory(screen, font, small_font, player):
    background_copy = screen.copy()

    inv_width = 700
    inv_height = 500
    inv_x = (1200 - inv_width) // 2
    inv_y = (600 - inv_height) // 2
    inv_items_x = inv_x + 30
    inv_items_y = inv_y + 230
    items_per_row = 5
    item_size = 64
    spacing = 10

```

```

inv_open = True
while inv_open:
    screen.blit(background_copy, (0, 0))
    overlay = pygame.Surface((1200, 600), pygame.SRCALPHA)
    overlay.fill((0, 0, 0, 180))
    screen.blit(overlay, (0, 0))
    panel = pygame.Surface((inv_width, inv_height))
    panel.fill((40, 40, 60))
    pygame.draw.rect(panel, (100, 100, 140), panel.get_rect(), 3)
    screen.blit(panel, (inv_x, inv_y))

    title_font = pygame.font.Font(None, 48)
    title = title_font.render("ИНВЕНТАРЬ", True, (255, 215, 0))
    screen.blit(title, (inv_x + inv_width // 2 - title.get_width() // 2, inv_y +
10))
    pygame.draw.line(screen, (150, 150, 180), (inv_x + 20, inv_y + 60), (inv_x +
inv_width - 20, inv_y + 60), 2)

    eq_x = inv_x + 30
    eq_y = inv_y + 80
    eq_title = font.render("Экипировка:", True, (255, 255, 200))
    screen.blit(eq_title, (eq_x, eq_y))

    weapon_rect = pygame.Rect(eq_x, eq_y + 30, 80, 80)
    pygame.draw.rect(screen, (60, 60, 80), weapon_rect, border_radius=8)
    pygame.draw.rect(screen, (200, 200, 220), weapon_rect, 2, border_radius=8)
    if player.equipped_weapon:
        icon = player.equipped_weapon.get_icon(64)
        screen.blit(icon, (weapon_rect.x + 8, weapon_rect.y + 8))
        weapon_text = small_font.render(player.equipped_weapon.name, True, (255,
255, 255))
        screen.blit(weapon_text, (weapon_rect.x + 5, weapon_rect.y + 85))
    else:
        empty_text = small_font.render("Пусто", True, (150, 150, 150))
        screen.blit(empty_text, (weapon_rect.x + 20, weapon_rect.y + 30))

    armor_rect = pygame.Rect(eq_x + 100, eq_y + 30, 80, 80)
    pygame.draw.rect(screen, (60, 60, 80), armor_rect, border_radius=8)
    pygame.draw.rect(screen, (200, 200, 220), armor_rect, 2, border_radius=8)
    if player.equipped_armor:
        icon = player.equipped_armor.get_icon(64)
        screen.blit(icon, (armor_rect.x + 8, armor_rect.y + 8))
        armor_text = small_font.render(player.equipped_armor.name, True, (255,
255, 255))
        screen.blit(armor_text, (armor_rect.x + 5, armor_rect.y + 85))
    else:
        empty_text = small_font.render("Пусто", True, (150, 150, 150))
        screen.blit(empty_text, (armor_rect.x + 20, armor_rect.y + 30))

```

```

item_rects = []
if player.inventory:
    for idx, item in enumerate(player.inventory):
        row = idx // items_per_row
        col = idx % items_per_row
        item_rect = pygame.Rect(
            inv_items_x + col * (item_size + spacing),
            inv_items_y + row * (item_size + spacing + 20),
            item_size, item_size
        )
        item_rects.append((item_rect, item))
        pygame.draw.rect(screen, (50, 50, 70), item_rect, border_radius=8)
        pygame.draw.rect(screen, (180, 180, 220), item_rect, 2,
border_radius=8)
        icon = item.get_icon(item_size - 10)
        screen.blit(icon, (item_rect.x + 5, item_rect.y + 5))
        if idx < 5:
            num_text = small_font.render(str(idx + 1), True, (255, 255, 0))
            screen.blit(num_text, (item_rect.x + 5, item_rect.y + 5))
            name_text = small_font.render(item.name[:12], True, (220, 220, 255))
            screen.blit(name_text, (item_rect.x + 5, item_rect.y + item_size -
15))
        else:
            empty_text = font.render("Инвентарь пуст", True, (200, 200, 200))
            screen.blit(empty_text, (inv_x + inv_width // 2 - empty_text.get_width()
// 2, inv_items_y + 50))

            hint = small_font.render("Нажми I чтобы закрыть | Цифры 1-5 использовать |
Наведи мышь на предмет | R - снять экипировку", True, (200, 200, 200))
            screen.blit(hint, (inv_x + inv_width // 2 - hint.get_width() // 2, inv_y +
inv_height - 40))

mouse_pos = pygame.mouse.get_pos()
tooltip_item = None
unequip_hint = False
if weapon_rect.collidepoint(mouse_pos) and player.equipped_weapon:
    tooltip_item = player.equipped_weapon
    unequip_hint = True
elif armor_rect.collidepoint(mouse_pos) and player.equipped_armor:
    tooltip_item = player.equipped_armor
    unequip_hint = True
else:
    for rect, item in item_rects:
        if rect.collidepoint(mouse_pos):
            tooltip_item = item
            break

if tooltip_item:
    effect_str = tooltip_item.get_effect_string()
    tooltip_lines = [

```



```

        f"Название: {tooltip_item.name}",
        f"Тип: {tooltip_item.type}",
        f"Эффект: {effect_str}",
        f"Цена: {tooltip_item.value} золота"
    ]
    if unequip_hint:
        tooltip_lines.append("Нажми R чтобы снять")
    line_height = 18
    tooltip_height = len(tooltip_lines) * line_height + 10
    tooltip_width = 260
    tooltip_surface = pygame.Surface((tooltip_width, tooltip_height),
pygame.SRCALPHA)
    tooltip_surface.fill((0, 0, 0, 220))
    pygame.draw.rect(tooltip_surface, (255, 215, 0),
tooltip_surface.get_rect(), 2)
    y_offset = 5
    for line in tooltip_lines:
        line_surf = small_font.render(line, True, (255, 255, 200))
        tooltip_surface.blit(line_surf, (5, y_offset))
        y_offset += line_height
    tx = mouse_pos[0] + 15
    ty = mouse_pos[1] + 15
    if tx + tooltip_width > 1200:
        tx = mouse_pos[0] - tooltip_width - 15
    if ty + tooltip_height > 600:
        ty = mouse_pos[1] - tooltip_height - 15
    screen.blit(tooltip_surface, (tx, ty))

pygame.display.flip()

for event in pygame.event.get():
    if event.type == pygame.QUIT:
        pygame.quit()
        sys.exit()
    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_i:
            inv_open = False
        if event.key == pygame.K_r:
            mouse_pos = pygame.mouse.get_pos()
            if weapon_rect.collidepoint(mouse_pos) and
player.equipped_weapon:
                player.unequip('weapon')
                show_dialogue(screen, font, "Снято оружие", "Инвентарь")
                inv_open = False
            elif armor_rect.collidepoint(mouse_pos) and
player.equipped_armor:
                player.unequip('armor')
                show_dialogue(screen, font, "Снята броня", "Инвентарь")
                inv_open = False

```

```

        if event.key in [pygame.K_1, pygame.K_2, pygame.K_3, pygame.K_4,
pygame.K_5]:
            idx = event.key - pygame.K_1
            if idx < len(player.inventory):
                item = player.inventory[idx]
                if player.use_item(item):
                    if item.type == 'potion':
                        player.inventory.pop(idx)
                        show_dialogue(screen, font, f"Использовано: {item.name}",
"Инвентарь")
                        inv_open = False
                        pygame.time.wait(50)

def show_battle_inventory(screen, font, small_font, player):
    background_copy = screen.copy()
    inv_width = 500
    inv_height = 400
    inv_x = (1200 - inv_width) // 2
    inv_y = (600 - inv_height) // 2

    usable_items = [item for item in player.inventory if item.type == 'potion']
    if not usable_items:
        overlay = pygame.Surface((1200, 600), pygame.SRCALPHA)
        overlay.fill((0, 0, 0, 180))
        screen.blit(overlay, (0, 0))
        msg = small_font.render("Нет предметов для использования в бою!", True, (255,
255, 255))
        screen.blit(msg, (1200//2 - msg.get_width()//2, 300))
        pygame.display.flip()
        pygame.time.wait(1500)
        return None

    selected_idx = 0
    inv_open = True
    while inv_open:
        screen.blit(background_copy, (0, 0))
        overlay = pygame.Surface((1200, 600), pygame.SRCALPHA)
        overlay.fill((0, 0, 0, 180))
        screen.blit(overlay, (0, 0))

        panel = pygame.Surface((inv_width, inv_height))
        panel.fill((40, 40, 60))
        pygame.draw.rect(panel, (100, 100, 140), panel.get_rect(), 3)
        screen.blit(panel, (inv_x, inv_y))

        title_font = pygame.font.Font(None, 48)
        title = title_font.render("ВЫБЕРИТЕ ПРЕДМЕТ", True, (255, 215, 0))
        screen.blit(title, (inv_x + inv_width // 2 - title.get_width() // 2, inv_y +
10))

```

```

y_offset = inv_y + 80
for idx, item in enumerate(usable_items):
    color = (255, 255, 255) if idx != selected_idx else (255, 255, 0)
    icon = item.get_icon(32)
    screen.blit(icon, (inv_x + 30, y_offset))
    text = font.render(f"{item.name} - {item.desc}", True, color)
    screen.blit(text, (inv_x + 80, y_offset + 5))
    num_text = small_font.render(str(idx+1), True, (255, 255, 0))
    screen.blit(num_text, (inv_x + 30, y_offset))
    y_offset += 50

    hint = small_font.render("Цифры 1-9 для выбора, ESC - отмена", True, (200,
200, 200))
    screen.blit(hint, (inv_x + inv_width // 2 - hint.get_width() // 2, inv_y +
inv_height - 40))

pygame.display.flip()

for event in pygame.event.get():
    if event.type == pygame.QUIT:
        pygame.quit()
        sys.exit()
    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_ESCAPE:
            return None
        if event.key in [pygame.K_1, pygame.K_2, pygame.K_3, pygame.K_4,
pygame.K_5, pygame.K_6, pygame.K_7, pygame.K_8, pygame.K_9]:
            idx = event.key - pygame.K_1
            if idx < len(usable_items):
                item = usable_items[idx]
                if item.use(player):
                    player.inventory.remove(item)
                    return f"Вы использовали {item.name}!"
                else:
                    return f"Не удалось использовать {item.name}!"
pygame.time.wait(50)

def show_chest(screen, font, small_font, player, chest_items, chest_rect):
    background_copy = screen.copy()
    chest_width = 500
    chest_height = 400
    chest_x = (1200 - chest_width) // 2
    chest_y = (600 - chest_height) // 2

    chest_open = True
    while chest_open:
        screen.blit(background_copy, (0, 0))
        overlay = pygame.Surface((1200, 600), pygame.SRCALPHA)
        overlay.fill((0, 0, 0, 180))
        screen.blit(overlay, (0, 0))

```

```

panel = pygame.Surface((chest_width, chest_height))
panel.fill((40, 40, 60))
pygame.draw.rect(panel, (100, 100, 140), panel.get_rect(), 3)
screen.blit(panel, (chest_x, chest_y))
title_font = pygame.font.Font(None, 48)
title = title_font.render("СУНДУК", True, (255, 215, 0))
screen.blit(title, (chest_x + chest_width // 2 - title.get_width() // 2,
chest_y + 10))
pygame.draw.line(screen, (150, 150, 180), (chest_x + 20, chest_y + 60),
(chest_x + chest_width - 20, chest_y + 60), 2)

if not chest_items:
    empty_text = font.render("Сундук пуст", True, (200, 200, 200))
    screen.blit(empty_text, (chest_x + chest_width // 2 -
empty_text.get_width() // 2, chest_y + chest_height // 2))
else:
    items_per_row = 2
    item_size = 80
    spacing = 20
    start_x = chest_x + 40
    start_y = chest_y + 100
    item_rects = []
    for idx, item in enumerate(chest_items):
        row = idx // items_per_row
        col = idx % items_per_row
        item_rect = pygame.Rect(
            start_x + col * (item_size + spacing),
            start_y + row * (item_size + spacing + 30),
            item_size, item_size
        )
        item_rects.append((item_rect, item))
        pygame.draw.rect(screen, (50, 50, 70), item_rect, border_radius=8)
        pygame.draw.rect(screen, (180, 180, 220), item_rect, 2,
border_radius=8)
        icon = item.get_icon(item_size - 10)
        screen.blit(icon, (item_rect.x + 5, item_rect.y + 5))
        num_text = small_font.render(str(idx + 1), True, (255, 255, 0))
        screen.blit(num_text, (item_rect.x + 5, item_rect.y + 5))
        name_text = small_font.render(item.name[:12], True, (220, 220, 255))
        screen.blit(name_text, (item_rect.x + 5, item_rect.y + item_size -
15))

mouse_pos = pygame.mouse.get_pos()
for rect, item in item_rects:
    if rect.collidepoint(mouse_pos):
        effect_str = item.get_effect_string()
        tooltip_lines = [
            f"Название: {item.name}",
            f"Тип: {item.type}",
            f"Эффект: {effect_str}",

```

```

        f"Цена: {item.value} золота"
    ]
    tooltip_surface = pygame.Surface((250, 80), pygame.SRCALPHA)
    tooltip_surface.fill((0, 0, 0, 220))
    pygame.draw.rect(tooltip_surface, (255, 215, 0),
tooltip_surface.get_rect(), 2)
    y_offset = 5
    for line in tooltip_lines:
        line_surf = small_font.render(line, True, (255, 255, 200))
        tooltip_surface.blit(line_surf, (5, y_offset))
        y_offset += 18
    tx = mouse_pos[0] + 15
    ty = mouse_pos[1] + 15
    if tx + tooltip_surface.get_width() > 1200:
        tx = mouse_pos[0] - tooltip_surface.get_width() - 15
    if ty + tooltip_surface.get_height() > 600:
        ty = mouse_pos[1] - tooltip_surface.get_height() - 15
    screen.blit(tooltip_surface, (tx, ty))
    break

    hint = small_font.render("ESC - закрыть | Цифры 1-3 - взять предмет | Наведи
мышь на предмет", True, (200, 200, 200))
    screen.blit(hint, (chest_x + chest_width // 2 - hint.get_width() // 2,
chest_y + chest_height - 40))
    pygame.display.flip()

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_ESCAPE:
                chest_open = False
            if event.key in [pygame.K_1, pygame.K_2, pygame.K_3]:
                idx = event.key - pygame.K_1
                if idx < len(chest_items):
                    item = chest_items.pop(idx)
                    if item.type == 'gold':
                        player.gold += item.effect
                        show_dialogue(screen, font, f"Вы нашли {item.effect}
золота!", "Сундук")
                    else:
                        player.inventory.append(item)
                        show_dialogue(screen, font, f"Вы взяли {item.name} из
сундука", "Сундук")
                chest_open = False
            pygame.time.wait(50)

def trade_with_trader(screen, font, small_font, player, trader):
    background_copy = screen.copy()

```

```

shop_width = 600
shop_height = 450
shop_x = (1200 - shop_width) // 2
shop_y = (600 - shop_height) // 2

goods = [
    Item("Зелье здоровья", "Восстанавливает 30 HP", "potion", 10, 30,
"assets/icons/potion.png"),
    Item("Стальной меч", "+5 к атаке", "weapon", 50, 5,
"assets/icons/st_sword.png"),
    Item("Кожаный доспех", "+3 к защите", "armor", 40, 3,
"assets/icons/armor.png")
]

trading = True
selected = 0
item_rects = []

while trading:
    screen.blit(background_copy, (0, 0))
    overlay = pygame.Surface((1200, 600), pygame.SRCALPHA)
    overlay.fill((0, 0, 0, 180))
    screen.blit(overlay, (0, 0))
    panel = pygame.Surface((shop_width, shop_height))
    panel.fill((40, 40, 60))
    pygame.draw.rect(panel, (100, 100, 140), panel.get_rect(), 3)
    screen.blit(panel, (shop_x, shop_y))

    title_font = pygame.font.Font(None, 48)
    title = title_font.render("ЛАВКА ТОРГОВЦА", True, (255, 215, 0))
    screen.blit(title, (shop_x + shop_width // 2 - title.get_width() // 2, shop_y
+ 10))
    pygame.draw.line(screen, (150, 150, 180), (shop_x + 20, shop_y + 60), (shop_x
+ shop_width - 20, shop_y + 60), 2)

    gold_text = font.render(f"Ваше золото: {player.gold}", True, (255, 215, 0))
    screen.blit(gold_text, (shop_x + 20, shop_y + 80))

    start_y = shop_y + 130
    item_rects = []
    selected_rect = pygame.Rect(shop_x + 20, start_y + selected * 70 - 5,
shop_width - 40, 70)
    highlight = pygame.Surface((selected_rect.width, selected_rect.height),
pygame.SRCALPHA)
    highlight.fill((255, 215, 0, 70))
    screen.blit(highlight, selected_rect)

    for i, item in enumerate(goods):
        icon = item.get_icon(48)
        screen.blit(icon, (shop_x + 30, start_y + i * 70))

```

```

name_text = font.render(f"{item.name}", True, (255, 255, 255))
screen.blit(name_text, (shop_x + 90, start_y + i * 70 + 5))
price_text = small_font.render(f"Цена: {item.value} золота", True, (255,
255, 200))
screen.blit(price_text, (shop_x + 90, start_y + i * 70 + 30))
desc_text = small_font.render(item.desc, True, (200, 200, 200))
screen.blit(desc_text, (shop_x + 90, start_y + i * 70 + 45))
rect = pygame.Rect(shop_x + 20, start_y + i * 70 - 5, shop_width - 40,
70)
item_rects.append((rect, item))

mouse_pos = pygame.mouse.get_pos()
for rect, item in item_rects:
    if rect.collidepoint(mouse_pos):
        effect_str = item.get_effect_string()
        tooltip_lines = [
            f"Название: {item.name}",
            f"Тип: {item.type}",
            f"Эффект: {effect_str}",
            f"Цена: {item.value} золота"
        ]
        tooltip_surface = pygame.Surface((260, 90), pygame.SRCALPHA)
        tooltip_surface.fill((0, 0, 0, 220))
        pygame.draw.rect(tooltip_surface, (255, 215, 0),
tooltip_surface.get_rect(), 2)
        y_offset = 5
        for line in tooltip_lines:
            line_surf = small_font.render(line, True, (255, 255, 200))
            tooltip_surface.blit(line_surf, (5, y_offset))
            y_offset += 18
        tx = mouse_pos[0] + 15
        ty = mouse_pos[1] + 15
        if tx + tooltip_surface.get_width() > 1200:
            tx = mouse_pos[0] - tooltip_surface.get_width() - 15
        if ty + tooltip_surface.get_height() > 600:
            ty = mouse_pos[1] - tooltip_surface.get_height() - 15
        screen.blit(tooltip_surface, (tx, ty))
        break

hint = small_font.render("W/S - выбор, SPACE - купить, ESC - выйти", True,
(200, 200, 200))
screen.blit(hint, (shop_x + shop_width // 2 - hint.get_width() // 2, shop_y +
shop_height - 40))
pygame.display.flip()

for event in pygame.event.get():
    if event.type == pygame.QUIT:
        pygame.quit()
        sys.exit()
    if event.type == pygame.KEYDOWN:

```

```

        if event.key == pygame.K_ESCAPE:
            trading = False
        if event.key == pygame.K_w:
            selected = (selected - 1) % len(goods)
        if event.key == pygame.K_s:
            selected = (selected + 1) % len(goods)
        if event.key == pygame.K_SPACE:
            item = goods[selected]
            if player.gold >= item.value:
                player.gold -= item.value
                player.inventory.append(item)
                show_dialogue(screen, font, f"Вы купили {item.name} за
{item.value} золота", "Торговля")
                trading = False
            else:
                show_dialogue(screen, font, "Не хватает золота!", "Торговля")
                trading = False
    pygame.time.wait(50)

def start_combat(screen, font, small_font, player, enemy):
    BG_COLOR = (30, 30, 50)
    PANEL_COLOR = (20, 20, 35)
    HP_BAR_BG = (80, 80, 80)
    HP_BAR_FILL = (255, 80, 80)
    TEXT_COLOR = (255, 255, 255)
    GOLD_COLOR = (255, 215, 0)
    MENU_BG = (40, 40, 60)
    MENU_BORDER = (100, 100, 140)

    combat = True
    last_attack_time = 0
    attack_delay = 0.5
    selected_option = 0
    menu_options = ["АТАКА", "РЮКЗАК", "СТАТУС", "УБЕЖАТЬ"]
    dialogue_text = "Что будем делать?"
    message_timer = 0

    enemy_max_hp = enemy.max_hp

    while combat:
        current_time = time.time()
        screen.fill(BG_COLOR)

        # ---- КАРТОЧКА ВРАГА (слева, y=20) ----
        enemy_panel_rect = pygame.Rect(20, 20, 500, 140)
        pygame.draw.rect(screen, PANEL_COLOR, enemy_panel_rect, border_radius=10)
        pygame.draw.rect(screen, MENU_BORDER, enemy_panel_rect, 2, border_radius=10)

        enemy_sprite = pygame.transform.scale(enemy.sprite, (120, 120))
        screen.blit(enemy_sprite, (enemy_panel_rect.x + 20, enemy_panel_rect.y + 10))

```



```

    enemy_name = font.render(enemy.name, True, TEXT_COLOR)
    screen.blit(enemy_name, (enemy_panel_rect.x + 150, enemy_panel_rect.y + 20))
    hp_text = small_font.render(f"HP: {enemy.hp}/{enemy_max_hp}", True,
TEXT_COLOR)
    screen.blit(hp_text, (enemy_panel_rect.x + 150, enemy_panel_rect.y + 50))
    hp_ratio = enemy.hp / enemy_max_hp
    bar_width = 200
    bar_height = 12
    fill_width = int(bar_width * hp_ratio)
    pygame.draw.rect(screen, HP_BAR_BG, (enemy_panel_rect.x + 150,
enemy_panel_rect.y + 70, bar_width, bar_height), border_radius=5)
    pygame.draw.rect(screen, HP_BAR_FILL, (enemy_panel_rect.x + 150,
enemy_panel_rect.y + 70, fill_width, bar_height), border_radius=5)

# ---- КАРТОЧКА ИГРОКА (справа, y=250) ----
player_panel_rect = pygame.Rect(screen.get_width() - 520, 250, 500, 140)
pygame.draw.rect(screen, PANEL_COLOR, player_panel_rect, border_radius=10)
pygame.draw.rect(screen, MENU_BORDER, player_panel_rect, 2, border_radius=10)

if player.idle_anim and player.direction in player.idle_anim:
    sprite_surf = player.idle_anim[player.direction][0]
    player_sprite = pygame.transform.scale(sprite_surf, (120, 120))
else:
    player_sprite = pygame.Surface((120, 120))
    player_sprite.fill((100, 100, 200))
screen.blit(player_sprite, (player_panel_rect.x + 360, player_panel_rect.y +
10))

player_name = font.render(player.name, True, TEXT_COLOR)
screen.blit(player_name, (player_panel_rect.x + 20, player_panel_rect.y +
20))
hp_text_player = small_font.render(f"HP: {player.hp}/{player.max_hp}", True,
TEXT_COLOR)
screen.blit(hp_text_player, (player_panel_rect.x + 20, player_panel_rect.y +
50))
hp_ratio_player = player.hp / player.max_hp
fill_width_player = int(bar_width * hp_ratio_player)
pygame.draw.rect(screen, HP_BAR_BG, (player_panel_rect.x + 20,
player_panel_rect.y + 70, bar_width, bar_height), border_radius=5)
pygame.draw.rect(screen, HP_BAR_FILL, (player_panel_rect.x + 20,
player_panel_rect.y + 70, fill_width_player, bar_height), border_radius=5)

# ---- МЕНЮ ДЕЙСТВИЙ ----
menu_width = 720
menu_height = 80
menu_x = (screen.get_width() - menu_width) // 2
menu_y = screen.get_height() - menu_height - 20
menu_rect = pygame.Rect(menu_x, menu_y, menu_width, menu_height)
pygame.draw.rect(screen, MENU_BG, menu_rect, border_radius=10)

```

```

pygame.draw.rect(screen, MENU_BORDER, menu_rect, 2, border_radius=10)

button_width = (menu_width // 4) - 15
button_height = 50
button_y = menu_y + (menu_height - button_height) // 2
for i, option in enumerate(menu_options):
    button_x = menu_x + 10 + i * (button_width + 10)
    button_rect = pygame.Rect(button_x, button_y, button_width,
button_height)
    pygame.draw.rect(screen, (60, 60, 80), button_rect, border_radius=8)
    if i == selected_option:
        highlight = pygame.Surface((button_width, button_height),
pygame.SRCALPHA)
        pygame.draw.rect(highlight, (255, 215, 0, 100), highlight.get_rect(),
border_radius=8)
        screen.blit(highlight, button_rect)
        pygame.draw.rect(screen, GOLD_COLOR, button_rect, 2, border_radius=8)
    else:
        pygame.draw.rect(screen, (150, 150, 180), button_rect, 2,
border_radius=8)
        text_surf = font.render(option, True, TEXT_COLOR)
        text_rect = text_surf.get_rect(center=button_rect.center)
        screen.blit(text_surf, text_rect)

# ---- ДИАЛОГОВОЕ ОКНО ----
dialog_rect = pygame.Rect(20, menu_y - 90, screen.get_width() - 40, 70)
pygame.draw.rect(screen, (0, 0, 0, 200), dialog_rect, border_radius=10)
pygame.draw.rect(screen, (200, 200, 200), dialog_rect, 2, border_radius=10)
dialog_surf = font.render(dialogue_text, True, TEXT_COLOR)
screen.blit(dialog_surf, (dialog_rect.x + 20, dialog_rect.y + 25))

pygame.display.flip()

# ---- ОБРАБОТКА ВВОДА ----
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        pygame.quit()
        sys.exit()
    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_a:
            selected_option = (selected_option - 1) % len(menu_options)
        elif event.key == pygame.K_d:
            selected_option = (selected_option + 1) % len(menu_options)
        elif event.key == pygame.K_SPACE:
            if selected_option == 0: # АТАКА
                if current_time - last_attack_time >= attack_delay:
                    last_attack_time = current_time
                    dmg = max(1, player.attack - enemy.defense)
                    enemy.hp -= dmg
                    dialogue_text = f"Вы нанесли {dmg} урона!"

```

```

        if enemy.hp <= 0:
            player.gold += enemy.gold_reward
            player.gain_experience(enemy.exp_reward)
            dialogue_text = f"Победа! +{enemy.exp_reward} опыта,
+{enemy.gold_reward} золота"

            for q in player.active_quests:
                if q.name == "Очистить поле": # проверка по точному
имени квеста
                    q.update_objective(0, 1)
                    if q.completed:
                        dialogue_text += f" Квест '{q.name}' выполнен!"
                        q.claim_reward(player)
                    pygame.display.flip()
                    pygame.time.wait(2000)
                    return True
            dmg_e = max(1, enemy.attack - player.defense)
            player.hp -= dmg_e
            dialogue_text += f" Враг атакует! -{dmg_e} HP"
            if player.hp <= 0:
                dialogue_text = "Вы погибли..."
                pygame.display.flip()
                pygame.time.wait(2000)
                pygame.quit()
                sys.exit()
        elif selected_option == 1: # ПЮКЗАК
            result = show_battle_inventory(screen, font, small_font,
player)

            if result:
                dialogue_text = result
                message_timer = current_time + 2.0
            else:
                dialogue_text = "Ничего не использовано."
                message_timer = current_time + 1.5
        elif selected_option == 2: # СТАТУС (врага)
            status_str = f"Враг: {enemy.name} | HP:
{enemy.hp}/{enemy.max_hp} | Атака: {enemy.attack} | Защита: {enemy.defense}"
            dialogue_text = status_str
            message_timer = current_time + 2.0
        elif selected_option == 3: # УБЕЖАТЬ
            dialogue_text = "Вы убежали!"
            pygame.display.flip()
            pygame.time.wait(1500)
            enemy.hp = enemy_max_hp
            return False

    if message_timer > 0 and current_time > message_timer:
        dialogue_text = "Что будем делать?"
        message_timer = 0

    pygame.time.wait(50)

```

```

# ===== ОСНОВНАЯ ФУНКЦИЯ =====
def run():
    pygame.init()
    screen = pygame.display.set_mode((1200, 600))
    clock = pygame.time.Clock()
    pygame.display.set_caption("RPG Game")
    font = pygame.font.Font(None, 32)
    small_font = pygame.font.Font(None, 20)

    # Фон
    try:
        back = pygame.image.load("background.jpg").convert()
        back = pygame.transform.scale(back, (1200, 600))
    except:
        back = None

    # Иконка золота для HUD
    try:
        gold_hud_icon = pygame.image.load("assets/icons/coin.png").convert_alpha()
        gold_hud_icon = pygame.transform.scale(gold_hud_icon, (32, 32))
    except:
        try:
            gold_hud_icon =
pygame.image.load("assets/icons/gold.png").convert_alpha()
            gold_hud_icon = pygame.transform.scale(gold_hud_icon, (32, 32))
        except:
            gold_hud_icon = None

    # Монета на карте
    coin_rect = pygame.Rect(500, 200, 40, 40)
    coin_taken = False
    try:
        coin_icon = pygame.image.load("assets/icons/gold.png").convert_alpha()
        coin_icon = pygame.transform.scale(coin_icon, (40, 40))
    except:
        coin_icon = None

    player = Player(100, 100, scale=1.2)

    # Сундук
    chest_items = [
        Item("Зелье здоровья", "Восст. 30 HP", "potion", 10, 30,
"assets/icons/potion.png"),
        Item("Старый меч", "+2 к атаке", "weapon", 20, 2, "assets/icons/sword.png"),
        Item("Золотая монета", "Добавляет 5 золота", "gold", 5, 5,
"assets/icons/gold.png")
    ]
    chest_rect = pygame.Rect(300, 200, 60, 50)
    try:

```

```

chest_img = pygame.image.load("assets/chest.jpg").convert_alpha()
chest_img = pygame.transform.scale(chest_img, (60, 50))
except:
    chest_img = pygame.Surface((60, 50))
    chest_img.fill((139, 69, 19))

# NPC (три крысы)
npc_list = []
trader = NPC(800, 300, "Торговец", "assets/trader.png", scale=1.6)
trader.add_dialogue("Хочешь что-то купить? Нажми Е ещё раз для торговли.")
npc_list.append(trader)

elder = NPC(900, 400, "Старейшина", "assets/elder.png", scale=1.6)
elder.add_dialogue("Очисти поле от крыс! Я награжу.")
npc_list.append(elder)

rat1 = NPC(700, 100, "Крыса", "assets/rat.png", is_enemy=True, scale=0.6)
npc_list.append(rat1)
rat2 = NPC(400, 400, "Крыса", "assets/rat.png", is_enemy=True, scale=0.6)
npc_list.append(rat2)
rat3 = NPC(800, 500, "Крыса", "assets/rat.png", is_enemy=True, scale=0.6)
npc_list.append(rat3)

# Квест
kill_quest = Quest("Очистить поле", "Очистите поле от крыс", 50, 100)
kill_quest.add_objective("Убить крыс", 3)
player.active_quests.append(kill_quest)

# Стартовый предмет
player.inventory.append(Item("Зелье здоровья", "Восст. 30 HP", "potion", 10, 30,
"assets/icons/potion.png"))

show_player_inv = False
interact_cooldown = 0
paused = False
victory_shown = False

while True:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            sys.exit()
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_i and not paused:
                show_player_inv = not show_player_inv
            if event.key == pygame.K_ESCAPE:
                if show_player_inv:
                    show_player_inv = False
                elif not paused:
                    paused = True
            else:

```

```

        paused = False

    if not show_player_inv and not paused:
        keys = pygame.key.get_pressed()
        dx = dy = 0
        if keys[pygame.K_a]: dx = -player.speed
        if keys[pygame.K_d]: dx = player.speed
        if keys[pygame.K_w]: dy = -player.speed
        if keys[pygame.K_s]: dy = player.speed
        player.move(dx, dy)

        player.x = max(0, min(1200 - player.width, player.x))
        player.y = max(0, min(600 - player.height, player.y))
        player.rect.x = player.x
        player.rect.y = player.y

        keys = pygame.key.get_pressed()
        if keys[pygame.K_e] and interact_cooldown == 0:
            if player.rect.colliderect(chest_rect):
                show_chest(screen, font, small_font, player, chest_items,
chest_rect)

                interact_cooldown = 30
            for npc in npc_list:
                if player.rect.colliderect(npc.rect):
                    if npc.is_enemy:
                        start_combat(screen, font, small_font, player, npc)
                        if npc.hp <= 0:
                            npc_list.remove(npc)
                    else:
                        dialogue = npc.get_dialogue()
                        show_dialogue(screen, font, dialogue, npc.name)
                        if npc.name == "Торговец":
                            trade_with_trader(screen, font, small_font, player,
trader)

                            interact_cooldown = 30
            if interact_cooldown > 0:
                interact_cooldown -= 1

            if not coin_taken and player.rect.colliderect(coin_rect):
                coin_taken = True
                player.gold += 5
                show_dialogue(screen, font, "Вы нашли 5 золотых!", "Система")

        # Отрисовка мира
        if back:
            screen.blit(back, (0, 0))
        else:
            screen.fill((30, 30, 50))

        screen.blit(chest_img, (chest_rect.x, chest_rect.y))

```

```

    if not coin_taken:
        if coin_icon:
            screen.blit(coin_icon, coin_rect)
        else:
            pygame.draw.ellipse(screen, (255, 215, 0), coin_rect)

    for npc in npc_list:
        npc.draw(screen)

    player.draw(screen)

    # HUD
    bar_width = 400
    bar_height = 20
    health_percent = player.hp / player.max_hp
    fill_width = int(bar_width * health_percent)
    pygame.draw.rect(screen, (80, 80, 80), (20, 20, bar_width, bar_height),
border_radius=10)
    pygame.draw.rect(screen, (255, 0, 0), (20, 20, fill_width, bar_height),
border_radius=10)
    hp_text = font.render(f"{player.hp}/{player.max_hp}", True, (255, 255, 255))
    text_rect = hp_text.get_rect(center=(20 + bar_width // 2, 20 + bar_height //
2))
    screen.blit(hp_text, text_rect)

    if gold_hud_icon:
        screen.blit(gold_hud_icon, (20, 70))
        gold_value = font.render(str(player.gold), True, (255, 255, 0))
        screen.blit(gold_value, (60, 75))
    else:
        gold_text = font.render(f"Золото: {player.gold}", True, (255, 255, 0))
        screen.blit(gold_text, (20, 70))

    # Квест
    if player.active_quests and not player.active_quests[0].completed:
        q = player.active_quests[0]
        obj = q.objectives[0]
        quest_text = font.render(f"{q.name}: {obj['current']}/{obj['target']}",
True, (0, 255, 0))
        screen.blit(quest_text, (1200 - quest_text.get_width() - 20, 20))

    # Показ окна победы (только один раз)
    if player.active_quests and player.active_quests[0].completed and not
victory_shown:
        show_victory(screen, font)
        victory_shown = True

    if show_player_inv:
        show_inventory(screen, font, small_font, player)
        show_player_inv = False

```

```
    if paused:
        show_pause_menu(screen, font, small_font)
        paused = False

    pygame.display.flip()
    clock.tick(60)

if __name__ == "__main__":
    run()
```